

Computer Organization

Design a custom 16-bit Processor using Verilog HDL

As part of OBE paradigm, in this course, you are expected to learn complex concepts and principles through complex real-world problems. This problem will help the students acquire depth of knowledge (WP1) regarding the in-depth structure and working of a processor by implementing a simplified Processor using **H/W Descriptive Language (Verilog)**. The problem covers your CL04 (mapped to PL05), and strengthens knowledge profile WK5.

Problem Statement

Project requires you to design and implement a custom instruction set processor (defined below) using HDL Verilog. The custom processor is based on **Harvard Architecture** i.e., it has a separate Instruction memory (which is read only memory i.e., ROM) and a Data memory (which you can read/write). The processing element will contain a Register file (with **8 different registers** and some **special purpose registers (SPR)**) and an Arithmetic and Logic Unit (ALU) capable of performing specific tasks (defined in Table 2). An abstract level view of the processor is shown in Figure 1.

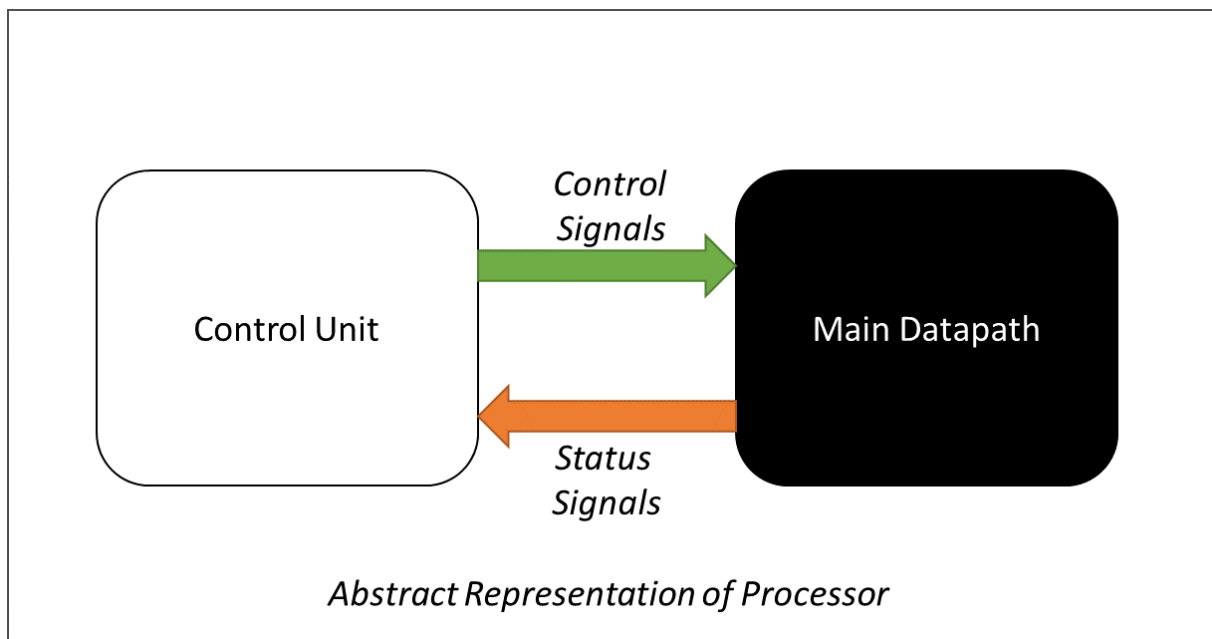


Figure 1 Abstract view of custom processor

The processor is divided into 2 parts (**Control Unit and Main Datapath**) which must be capable of performing a **basic set of instructions** given under Main Datapath (Details are discussed in **Main Datapath (Basic)**). Both control unit and datapath are connected using **Status Signals** and **Control Signals**.

In addition to the working of Main Datapath we intend to enhance the working of processor by adding Instructions which are Conditionally Execution (**Conditional execution means that condition flags control the ALU whether the instruction will be executed or not**). The additional instructions are specified under **Conditional Instructions** section.

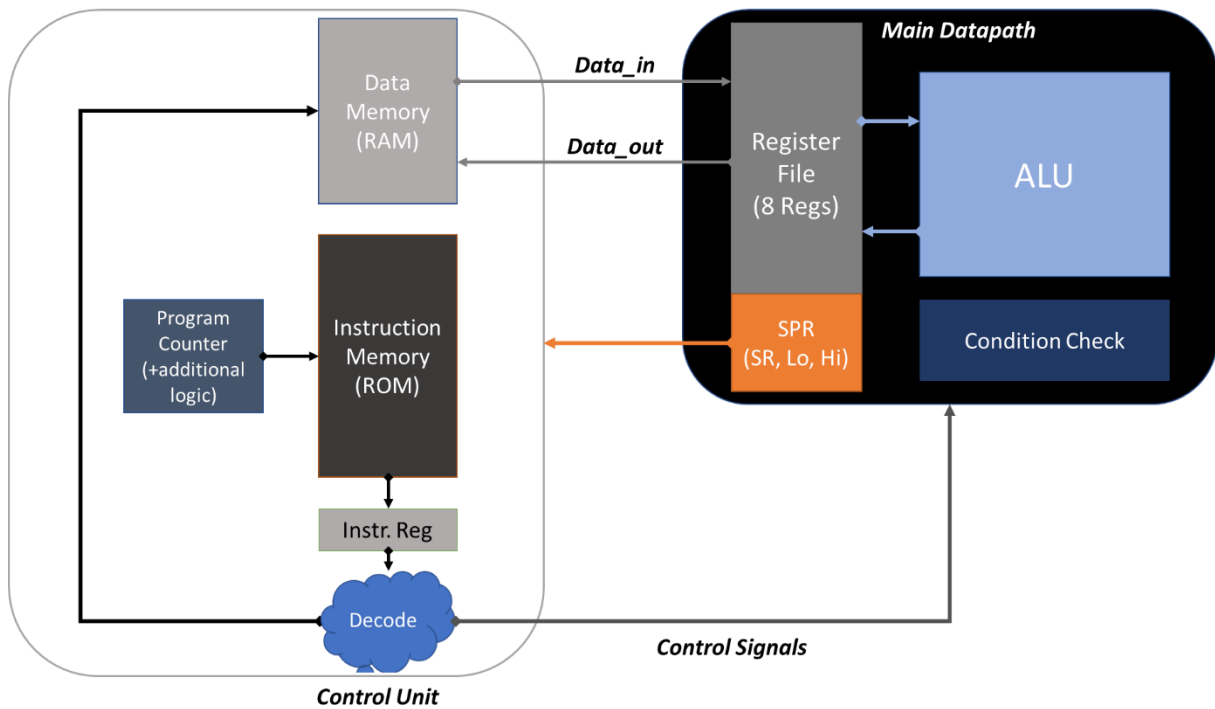


Figure 2 Detailed view of the processor

1. Main Datapath (Basic)

The Main processor must be able to execute all 3 types of instructions that we studied in MIPS (though with slight modification for ease of implementation).

1.1. Instruction Format (Main Basic CPU)

The Main processor must be able to execute all 3 types of instructions that we studied in MIPS (though with slight modification for ease). Instruction Format and types to be followed are as:

Table 1 Instruction Type and field details

Instruction Type	0-3	4-6	7-9	10-12	13-15
<i>R</i> (Register)	Opcode	Destination (R_d)	Source 1 (R_s)	Source 2 (R_t)	Shift Amount
<i>I</i> (Immediate)	Opcode	Destination (R_d)	Source (R_s)	Constant/Address	
<i>J</i> (Jump)	Opcode	Address			Unused

1.2. Instruction Set (Main Basic CPU)

Following is the set of basic instructions that your processor must be able to execute

Table 2 Instruction set along with details of each instruction

Instructions	Opcode	Type	Mathematically
$add\ R_d, R_s, R_t$	0000	R	$R_d = R_s + R_t$
$sll\ R_d, R_s, shamt$	0001	R	$R_d = R_s \ll shamt$
$slr\ R_d, R_s, shamt$	0010	R	$R_d = R_s \gg shamt$
$mul\ R_s, R_t$	1010	R	$[Hi, Lo] = R_s \times R_t$
$mflo\ R_d$	1011	R	$R_d = Lo$
$mfhi\ R_d$	1100	R	$R_d = Hi$
$or\ R_d, R_s, R_t$	0011	R	$R_d = R_s R_t$
$and\ R_d, R_s, R_t$	0100	R	$R_d = R_s \& R_t$

<i>addi</i> $R_d, R_s, constant$	0101	I	$R_d = R_s + constant$
<i>li</i> $R_d, constant$	0110	I	$R_d = constant$
<i>lw</i> $R_d, constant(R_s)$	0111	I	$R_d = memory[R_s + constant]$
<i>sw</i> $R_d, constant(R_s)$	1000	I	$memory[R_s + constant] = R_d$
<i>b</i> <i>address</i>	1001	J	$PC = PC + address$

You must build on the code done in lab sessions. Some other snippets of code will be shared with the class for ease of implementation. Your aim is to make this design work using HDL Verilog. Some important characteristics of the Processor are mentioned below:

1. All data registers and memory are **16-bit wide**
2. Instruction memory is a **Read-Only Memory (ROM)**
3. PC register is only **8-bit wide**, and the program/instructions start executing from address **0x00**
4. We can read/write the data memory
5. The main CPU has a special purpose register by the name of "Status Register (SR)" with the following configuration

Remaining bits are unused	Zero	Negative	Carry	Overflow
---------------------------	------	----------	-------	----------

Working:

sub \$t0,\$t1,\$t2

- Case 1:

$\$t1 = \$t2$

Zero flag (i.e., bit location for Zero flag) will be set to 1

- Case 2:

$\$t1 < \$t2$

Negative flag (i.e., bit location for Negative flag) will be set to 1

- Case 3:

If overflow occurs

Overflow flag (i.e., bit location for Overflow flag) will be set to 1

2. Conditional Instructions

We intend to add **special conditional instructions** which get executed by looking at flags set by previous command e.g.,

sub \$t0,\$t1,\$t2
bz exit #branch if zero flag is set

... .

exit:

Table 3 discusses the instructions that are to be added in the base processor. To implement these changes, review the basics of the processor and make amendments in appropriate places in the design.

Table 3 Conditional Instructions to be incorporated in the base processor

Instructions	Mathematically
$addx\ r_d, r_s, r_t$	Add if flag x is set ($x \in \{N, Z\}$)
$bx\ label$	Jump to <i>Label</i> if flag x is set ($x \in \{N, Z, OF, C\}$)

Notes:

- Project can be done in groups of 3 or less members.
- The report must clearly mention the contribution of each member to the project work.
- Marking of Project is based on 3 factors, "Demonstration (code)", "Documentation (Report)" and "Viva".
- You need to submit the project report and code one week after final exams.
 - Report must contain the design on paper/VISIO with complete details regarding instruction set.
- Demonstration along with VIVA will be conducted for individual marking.
- **Extra credit will be given to students who add additions instructions in the design**